

Final Year Project

Protein Language Models for Near-Contact Map Prediction

Ryan Koenig

Student ID: 22201444

A thesis submitted in part fulfilment of the degree of

BSc. (Hons.) in Computer Science

Supervisor: Gianluca Pollastri



UCD School of Computer Science

University College Dublin

April 26, 2026

Table of Contents

1	Project Specification	3
2	Introduction	4
3	Background	5
3.1	Protein Structures	5
3.2	Contact Maps	5
3.3	Multiple Sequence Alignments	6
3.4	Protein Language Models	7
3.5	Fully Convolutional Neural Networks	7
4	Goals and Time Management	9
5	Methods	11
5.1	Data Considerations	11
5.2	Implementation	13
6	Results	15
6.1	VHSE	15
6.2	ProtTrans	16
7	Conclusion	21

Abstract

Protein contact map prediction remains a challenge in bioinformatics, with a lot of potential improvements. For years bioinformatics has used the multiple sequence alignment as the primary form of encoding for predicting protein tertiary structure. With the resurgence of machine learning and the rise of large language models like ChatGPT, modern bioinformatics research has shown potential in leveraging these large language models for use in bioinformatics.

This project aims to examine the claims made by researchers like Ahmed Elnaggar, whose ProtTrans T5 model is claimed to be able to create protein embeddings at a far greater speed and the same if not better performance than the state-of-the-art multiple sequence embeddings. A supervised fully convolutional neural network was used in order to test the per-residue embeddings of the T5 model, examining performance, time complexity, and constraints.

The results indicated potential in the future of protein language models like ProtTrans. Embeddings were generated far faster than MSA analogs, and the accuracy of the embeddings performed as expected. However, these improvements came at the cost of extremely high embedding vector dimensionality which dramatically slowed down training. This suggests that while protein language models may not be a complete replacement for multiple sequence alignments, they will have their uses, and they show great potential if improvements can be made.

Chapter 1: Project Specification

The primary goal specified for this project is testing pretrained deep learning algorithms. This project should test the performance and time complexity of Ahmed Elnaggar's ProtTrans protein language models [1]. This project will need to report on performance compared to single protein sequences in both computation and time complexity, determining the potential impact and future for large language models in the realm of bioinformatics and novel protein structure prediction algorithms. This project will draw conclusions on the feasibility of these large language models as compared to the current standard, consisting of positional embeddings like PSI-BLAST and multiple sequence alignments, however, this project's scope does not account of independent testing of PSI-BLAST MSAs.

Chapter 2: Introduction

Proteins are a key feature in understanding the human body. Used to develop medicines and understand or prevent myriad diseases and afflictions. One of the most important parts of proteins is the shape they take, which is often what determines what these proteins do in our bodies. However, analyzing and determining the shape of a protein requires a great deal of resources and time.

Protein contact maps, a common way to represent protein shape, are often used to predict proteins structure where it is not known. (Fig. 3.1) The prediction of protein structures is a complex problem in bioinformatics. Neural networks, evolutionary models, and other forms of machine learning are often used in attempts to quickly discover protein shapes from their amino acid chains. With these forms of machine learning, one of the most important aspects of the model is how the residue chain of the protein is encoded.

With the rise of large language models in modern computing and machine learning, there have been many specialized use cases for LLMs. In 2021 Ahmed et al. published a set of pretrained large language models trained on protein structure information; stating that the ProtTrans models were faster to compute while matching or exceeding the performance of competing state-of-the-art embeddings. The 'T5' protein language model in particular was reported to perform significantly better when fed into relatively small convolutional neural networks, or feedforward neural networks.

This project aims to evaluate the feasibility by feeding the T5 protein language model embeddings into a fully convolutional neural network, testing its performance in creating these structure predictions. The T5 model will be tested on two fully convolutional neural networks at varying depths and compared to the performance of a much lower-detail embedding in order to evaluate the increase of performance of the protein language model embeddings.

Chapter 3: Background

3.1 Protein Structures

In order to best explain how proteins are shaped, I will summarize a more detailed article by Stoller [2]: Life on Earth is mainly composed of four macromolecules: nucleic acids (DNA, RNA), lipids, polysaccharides, and proteins. Proteins specifically are polymers of 30 or more molecules mostly composed of three parts: an amino group, an R-group, and a carboxylic acid group. When amino acids are polymerized by a ribosome and the corresponding mRNA, the chain is initially straight with 'tails' coming off the sides depending on the R-groups of the amino acids, and the now bonded amino acids are referred to as amino acid 'residues'.

The shape of any protein depends almost entirely on the chain of amino acids that form that protein (the 'primary structure' and the R-group's tails). Stoller uses sickle-cell anemia as an example to explain protein folding. In this case, simply replacing a single amino acid in the right position in hemoglobin leads the normal disk-shaped protein to instead fold into the nominal sickle shape. This is a great example of protein folding being mostly dependent on the primary structure of the protein. [3]

Through myriad molecular and atomic forces, the protein is able to spontaneously fold into a consistent shape, the protein's 'tertiary structure'. The complexity of these forces leads to the problem of determining a protein's shape being potentially NP-Hard [4]. With the difficulty and complexity involved in calculating the shape of a protein's tertiary structure, it is often much easier to simply experimentally analyze a protein. Neural networks, and other machine learning models, are able to recognize and see the patterns in protein primary structure that cannot be calculated, making them the current best way to predict protein tertiary structure.

3.2 Contact Maps

Proteins, as complex polymers, often have lengths that can well exceed hundreds of amino acids alongside complicated three dimensional structures. This raises a problem for representing the tertiary structures of individual proteins. A common solution for this is a 'contact map'. Contact maps are a two dimensional representation of the distances between each amino acid pair. In this project, we are interested in near-contact maps, which typically represent a distance of 8 angstroms or less as a 'contact'.

An example of a protein contact map: Fig. [3.1](#)

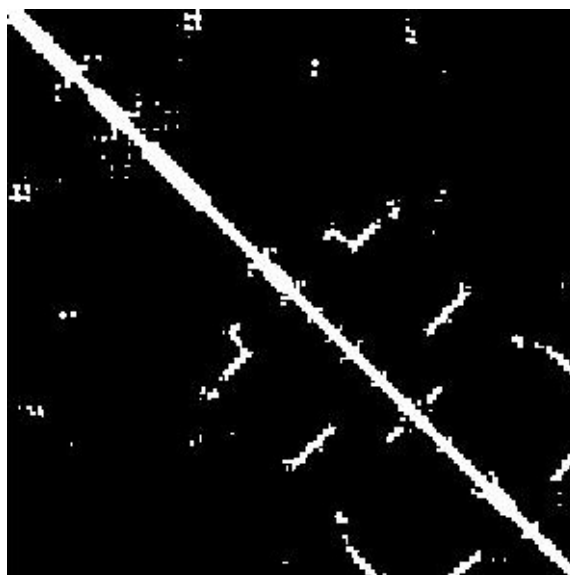


Figure 3.1: An example of a protein contact map, for a protein of length 200. White represents spots where two amino acid residues are within 8 angstroms of each other, or 'contacts.' Black represents residues that are not within 8 angstroms

3.3 Multiple Sequence Alignments

Proteins are often represented by the amino acids in their backbone through a string of characters (e.g. 'M A S I L D C P I V') where each character represents an amino acid. For machine learning algorithms in bioinformatics, this poses a problem: How do you convert the long string of residue characters into a machine-readable information dense format? The current answer is multiple sequence alignments (MSA). The Position-Specific Iterative Basic Search Alignment Tool, or 'PSI-BLAST', is one of the most common and successful of these tools.

MSAs, like PSI-BLAST, use an intensive algorithm to calculate Position-Specific Scoring Matrices, also known as a PSSM score. The PSI-BLAST algorithm works by taking an input amino acid sequence and comparing sequences of 3 to 11 amino acids against some or all of the proteins in a given protein database, then with the database hits, a multiple sequence alignment is generated. PSI-BLAST then uses these MSAs to calculate a score for each amino acid for each position in the original protein. These scores represent the positional favorability of all amino acids for a given position for each amino acid.

When the PSSM is used as training data in a CNN, it can provide important connections between shapes and groups of similar amino acids, which are likely to fold similarly. The primary issue with PSSMs is the time it takes to compute, as well as its inability to consider the whole protein, as it relies on analyzing it as multiple sequences. The computation time is especially devastating, as creating accurate PSSMs for a large dataset of proteins involves sweeping entire datasets for each protein, leading to extreme time complexity. These are the key problems protein language models aim to solve.

3.4 Protein Language Models

Protein language models, or pLMs, are a potential replacement for MSAs. A pLM performs similarly to other large language models. They both take in sentences that are broken into tokens for each word and fed into the LLM, with the LLM producing an output according to the input tokens. In most cases, like OpenAI's ChatGPT, the output consists of a prediction of what should follow those words in that language, usually in the form of an answer to a question or request. Protein language models work in much the same way as large language models, but instead of words in English or another language, the language of a pLM is proteins.

The sentence that pLMs take as an input is the primary sequence of the protein, the amino acid chain. ProtTrans, the pLM of interest to this project, is trained on known amino acid chains, where each residue is a word for the tokenizer and large language model [1]. In the case of ProtTrans residue embeddings, the output is a vector embedding of each amino acid that ideally contains enough structural information to be able to train a supervised network.

The primary object of pLMs like ProtTrans is to reduce the time spent creating the embeddings used. A properly trained pLM would be able to generate dozens of embeddings within a single second, far exceeding the speed of generating a multiple sequence alignment.

3.5 Fully Convolutional Neural Networks

With the encodings of the proteins, it is then necessary to create a trained model that can take any encoding and create a prediction. One of the most common machine learning models for contact maps specifically are neural networks. Neural networks are a model that consist of multiple layers of 'neurons' which feed a given input into the next layer of neurons depending on weights in an activation function. In training, each neuron's weight is adjusted by calculating the total error of the system, and working backward to determine how much to change each value by, typically repeated over and over alongside an optimization algorithm like gradient descent.

The 'depth' of a neural network is defined by how many layers the network uses. This is usually measured by how many layers are between in the input and output layers, with 'deep' networks often containing many more than 2 'hidden' layers. Often neural networks are used to classify or predict complicated nonlinear problems, deep neural networks are often able to amplify more subtle trends in the input data. Regarding protein tertiary structure and contact map prediction specifically, deep 'convolutional' neural networks have shown extra promise.

Convolutional neural networks (CNN) work very similar to regular neural networks. However, instead of taking inputs independently, convolutional layers take a filter of part of the input. These 'convolutions' give CNNs the ability to identify patterns and spatial structure in an input image, or 'tensor'. This ability to recognize larger-scale spatial patterns in data is what gives CNNs their promise in protein tertiary structure prediction. A pair-wise map of protein amino acid embeddings is, in essence, an extremely large image with very subtle spatial features.

There are a few ways to measure the performance of a neural network. For this project, there will be 4 numerical scores with a few visualizations of the predicted contact maps. The four numerical scores used are accuracy, precision, recall, and F1. All of these formulas rely on 'true' and 'false' combined with 'positive' or 'negative' predictions. For contact maps, a positive represents a prediction that those two amino acids will be in contact, with a negative representing no contact. True and false confirm whether the prediction was true or not. E.g., the count of 'true positive'

predictions is a count of how many times the neural network predicted that there was a contact and was correct. Due to the sparse nature of contact maps, recall and F1 are the most important scores for this project.

Chapter 4: Goals and Time Management

This project was initially split into two subsections, one per semester. The first section would cover the majority of the research. With very little knowledge on bioinformatics understanding proteins, learning common practices, and developing an idea for how to go about implementing the specifics of this project were all learned here through reading papers and analyzing similar projects. The largest problem with this project started here however, as I either overestimated my ability to learn PyTorch, or underestimated its complexity.

This leads into the second semester and second planned section for this project, the implementation. Work on code began in January, but was far slower than I had anticipated, as the PyTorch documentation did not prove particularly helpful in explaining how to create a CNN, a dataset, or a proper training loop. Eventually, I had an understanding of PyTorch that allowed me to create my first CNN to test protein data on. At this point however, the schoolwork of the semester had picked up and my available time to spend on the project had decreased significantly, leading to an overall delay in the project's implementation reaching completion.

This difficulty with time is without a doubt the largest flaw in this project, as it likely is in many projects. Neural networks can take a long time to train and optimize, especially on datasets with inputs this large. The specifics of how time was worked around through the implementation will be discussed in later chapters, but it proved the greatest limit in developing a more accurate CNN.

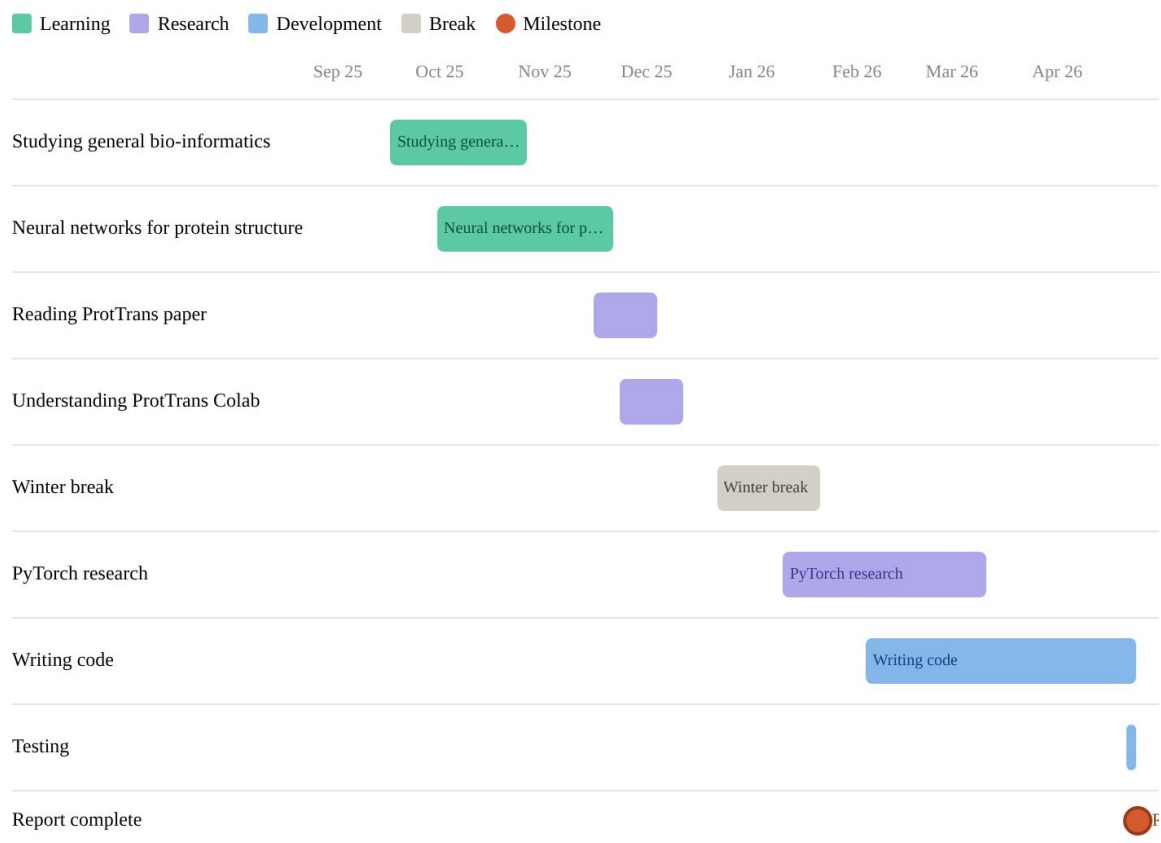


Figure 4.1: A gantt chart representation of the reality of time spent on this project. Some papers are generalized and detail into coding work is combined for simplicity. The time difficulties in the month of April are apparent. It is worth noting the difficulties are not due to a lack of time spent on it beforehand, the difficulty in the beginning caused a sudden finish, which came at the cost to the ability to continue experimenting with the implementation to get a better result.

Chapter 5: Methods

5.1 Data Considerations

5.1.1 Structure

The two files used for data consist of a FASTA file and dataset file, both of which are highly structured. The original protein sequences are taken from the full Protein Data Bank, a trustworthy, up-to-date source containing a vast majority of all of the currently known protein structures.

The FASTA is a simple file formatting standard used in biochemistry and bioinformatics. One line contains the protein name (IE. '>1A3N'), with the following line containing the primary structure of the protein. The dataset file contains much more detailed information about each residue, but contains the same proteins as the FASTA file. The relevant information in this file is the protein name, residue series, length, and positional coordinates. The positional coordinates in this file are what we can use to compute the contact map for a protein. Both of these files are stored locally, which allows for the dataset classes to process and store their information separately as needed.

5.1.2 Processing

The files have only been permanently processed once. Each protein sequence is compared pairwise aligned, where any protein with a thirty-percent or higher similarity was removed. This ensures no proteins are too similar, which would allow a neural network to memorize the pattern, rather than learn connections. After this reduction, the dataset contains over five hundred thousand proteins.

The FASTA and data set files are then read in and are processed, encoded, and stored in distinct files. There are two forms of encoding that will be used. The first is ProtTrans, the primary encoding of interest for this project. The second will serve as a control group to compare against ProtTrans. This second group will encode the protein residues using Vectors of Hydrophobic, Steric, and Electronic properties (VHSE). During both of the encoding processes, a max length of 200 is set for the proteins, as was done in similar experiments by Martin et al. [5] [6]. The primary purpose of this is to reduce the time to train the neural network. For a given length of a protein N with an encoding vector of length V , a given input tensor for a neural network is $N \times N \times V$. This rule makes the length of the protein the most influential aspect of the total time needed to train a neural network. Due to difficulties discussed later in this report, reducing the time spent training the neural network was necessary. This also has the side effect of reducing the dataset to a size of roughly 14,000 data points, split between 1000 testing proteins and just over 13000 training proteins.

The amino acid positional coordinate data is also processed into a tensor. Initially the structure of the coordinates consists of a three dimensional vector of doubles that corresponds to an X, Y, and Z value in angstroms. In order to make a contact map that can be used as the target for a neural network, a pairwise matrix is made. The difference between the two amino acid pair's coordinates is calculated, followed by taking the normal of the vector. This results in a double representation of the distance between the two amino acid pairs. Following this, a threshold is applied where any

value less than or equal to eight angstroms is replaced with a 1, denoting a contact, else it is a 0, denoting no contact.

5.1.3 ProtTrans Encoding

The ProtTrans encoded dataset is the key point of interest for this project. The encodings are performed using the ProtTrans 'T5' encoder, which is stated to be ProtTrans best pre-trained general model. The amino acid strings were then sorted and sent to the HuggingFace LLM. Of the returning data, this project uses the per-residue embeddings, which consist of a vector of length 1024 for every amino acid. This information is then transformed into a pair-wise square tensor with a width of each protein length. In order to best represent the pairwise combination of the two amino acids, the two embedding vectors for the amino acids are concatenated, followed by their product and their difference. This results in a very large per-pair vector of 4096.

There is no doubt that the large size of the vectors contributes to the majority of the training time for the ProtTrans dataset, and may not be necessary in a deeper or more advanced neural network. However, through experimentation, these pair-wise operations resulted in the highest recall and F1 score for the neural network compared to different operations or shorter vectors.

The time taken to generate the embeddings is one of the key features of ProtTrans. Each protein takes between 0.025 and 0.1 seconds to generate an embedding for, with a mean of roughly 0.055 seconds, which held linearly with the number of proteins embedded Fig. 5.1. This serves as an improvement over competing embeddings like PSI-BLAST, which can take far longer per-protein, as the T5 embedding model can entirely avoid querying potentially an entire protein database.

Encoding Time

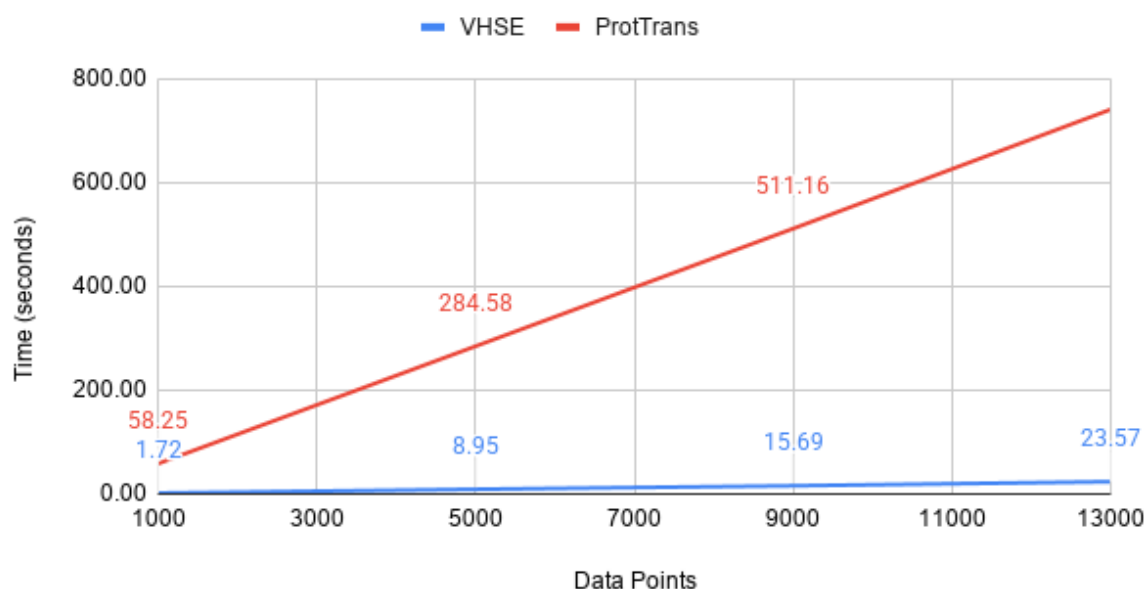


Figure 5.1: Time to encode a dataset of a given size for both encoding methods. VHSE encoding requires minimal time, and serves as a representation of the base compute time of the program. The time required per-protein stays linear as the dataset increases, with a per-protein time of 0.058 to 0.056 seconds through each data point.

5.1.4 VHSE Encoding

In order to properly measure the performance of the ProtTrans dataset, a control is required. As the goal of this project is to measure the performance and relative speed of ProtTrans, VHSE encodings serve as an important baseline for all measurements. The VHSE embeddings are not specially made for storing protein structure like ProtTrans or PSI-BLAST.

As the name suggests, VHSE embeddings mainly store information about the interactions of these amino acids with other molecules. Some of those attributes can help determine the tertiary structure of a protein. For this project, VHSE serves as a way to convert each amino acid residue into a unique and recognizable vector in a form similar to ProtTrans. The VHSE vectors are far smaller than the ProtTrans vectors, at a length of 8. However, when processed in the same manner as the ProtTrans vectors, to minimize uncontrolled variables, the associated vector of an amino acid residue has a length of 32. With a vector length similar to PSI-BLAST's 28, the time per training epoch of a neural network should be similar, serving as a point of comparison.

Calculating a VHSE embedding is also trivial, requiring little more than an iteration through the protein primary structure. This can be used to calculate the actual time the ProtTrans models require to generate embeddings without bias from the machine calling the API to create the embeddings.

5.2 Implementation

5.2.1 Datasets

The datasets for this project were both programmed in Python using PyTorch's Dataset class as a parent. These classes read in the proteins' primary structure and coordinate from the data set files, then the classes create both the embeddings and the contact map. This information is then stored in a file alongside the length of the protein. When either Dataset class is called to retrieve an item at index 'i' the dataset reads in that section of the file, then performs the vector operations to create the input tensor. The input tensor, target contact map, and length are then returned.

This method is likely unconventional for PyTorch Datasets. The reason for the unconventional implementation is hardware limitations. Through training iterations, the reading speed of an SSD as well as the processing speed of the CPU were the primary limitations, and multiple different approaches for minimizing their effects were taken, with this method's performance exceeding that of the other functioning iterations. There remains two potential improvements for this project that were not able to be implemented either due to hardware or time constraints.

The first improvement is theoretically simple, and is the most feasible for future improvements on the CNN and it's Dataset classes. To reduce the effect of the CPU on the training time of the CNN, increasing the number of workers sending data to the GPU could be a fairly simple fix. Unfortunately, as the machine running the training code is a home desktop with Windows 11 installed, this was more difficult than it is worth. Windows is not able to fork scripts in the same way a Linux computer can, and instead runs another version of this script. In practice, this means attempting to add more workers slowed down progress, as the scripts did not communicate with the same goal. This could be fixed by using a Linux operating system. Windows Subsystem for Linux was attempted, but due to permission restraints in the Linux Subsystem terminal, attempting to run a python script with the required libraries installed began consuming more time than would be gained by having just been running the training code during the attempt.

The second improvement was to attempt to pre-compute each input tensor, then store them as tensor files using PyTorch's built in save function. The idea of this improvement is to front-load all of the CPU computations, allowing epochs to run faster and match the speed of the GPU. The tradeoff was that the new limitation would be the read speed of the SSD. There were two separate issues with this caused by the same issue, the size of the input PyTorch tensors. In the worst case, a tensor could consist of over 163 million floating point numbers. Stored in half-precision floating point format, this results in a per-tensor storage space of a third of a gigabyte. Resulting in a total worst-case storage space requirement of over 4.5 terabytes for only 14 thousand data points. Aside from this amount of required storage space being infeasible given the hardware restrictions of the machine being used for training, reading in tensors of such a size inevitably slowed down the training more than the CPU's computations did.

In future work, increasing the maximum length of protein would greatly increase the amount of available data for the neural networks in this project. Due to the large increase in computation time that would follow that, the rather steep maximum length cut was kept.

5.2.2 Neural Networks

A neural network is one of the most common machine learning models for protein structure prediction. In the ProtTrans paper [1], Elnaggar specifically mentions convolutional neural networks and feedforward neural networks performing particularly well on the ProtTrans embeddings. Similarly David T. Jones showed that a sufficiently deep CNN is able to achieve high performance even when using shallow sequence alignments [7]. Accordingly, this project uses a fully convolutional neural network for classification.

A 'fully' convolutional network is defined by its use of only convolutional layers. Often when creating a CNN, the later layers will consist of normal neurons, requiring an input for every neuron. Using only convolutional layers, FCNs are able to take in input of variable length, and result in more complicated outputs. The variable length of an FCN is extremely useful for protein structure prediction as it allows the neural network to take in protein embedding tensors without padding, helping reduce sparsity in the already very sparse data of a contact map.

For this project, an FCN with 6 hidden layers was implemented using PyTorch's neural network library. Initially, a shallower FCN with a depth of 2 was trained and tested, but resulted in lower recall and F1 scores compared to the later 6-layer FCN. A non-convolutional regression neural network was also tested to attempt to predict distances instead of classify contact, but this resulted in a completely incorrect output tensor, and was not explored further.

Training the FCN was done through a combination of the Adam gradient descent optimizer and the Binary Cross Entropy (BCE) with logits loss function, both from PyTorch. Other combinations of optimizer and loss functions were attempted (e.g., stochastic gradient descent and BCE without logits) but performed worse than Adam and BCE with logits.

Chapter 6: Results

6.1 VHSE

The performance of the VHSE embedding trained FCN was poor, as expected with VHSE's lack of structural or positional values. Even the relatively deep FCN was unable to reliably predict contact maps. With the combination of the lack of information, and the sparsity of contacts away from the backbone of the protein, the FCN was only predicting contacts on the diagonal after only one epoch, see Figs. 6.1, 6.2.

Due to this loss of detail, the recall and F1 of the VHSE FCN are relatively low at a best score of 0.65 and 0.75 at its best in epoch 3, see Fig. 6.3. However, the accuracy and precision are remarkably high across all epochs at 0.96 and 0.9 for accuracy and precision respectively. This is not due to any feature of the VHSE FCN, it is simply the nature of scoring contact map prediction. These statistics serve as a control set for the ProtTrans trained CNN, where if both FCNs are only able to achieve these scores, it will be evidence that ProtTrans is not able to successfully capture tertiary structure data in their embeddings.

With the relatively small vector size of the VHSE embeddings, the training of the FCN is extremely fast, taking only 6 minutes per epoch. This is one of the largest upsides with the VHSE and PSI-BLAST embeddings, the small input tensor size allows for quick training on weaker systems once the embeddings have been generated. The majority of this time gain likely comes through making it easier to read and manipulate the tensors by the SSD and the CPU, allowing for the GPU to become the primary hardware bottleneck.

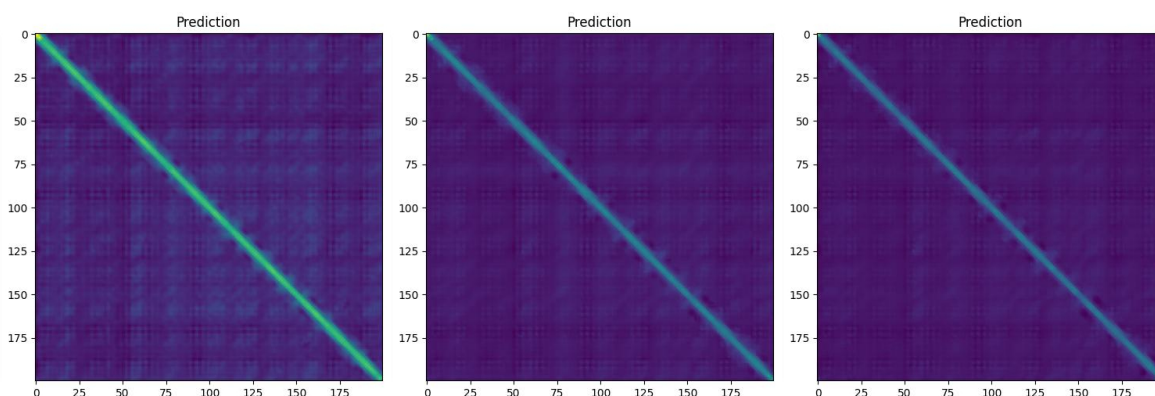


Figure 6.1: FCN Trained on VHSE prediction values at 1, 5, and 10 epochs. It is immediately evident by looking at the predictions that the vast majority of contact information is lost very quickly with the VHSE encodings. As a baseline comparison this is desired. It is, however, not useful for predicting protein tertiary structures.

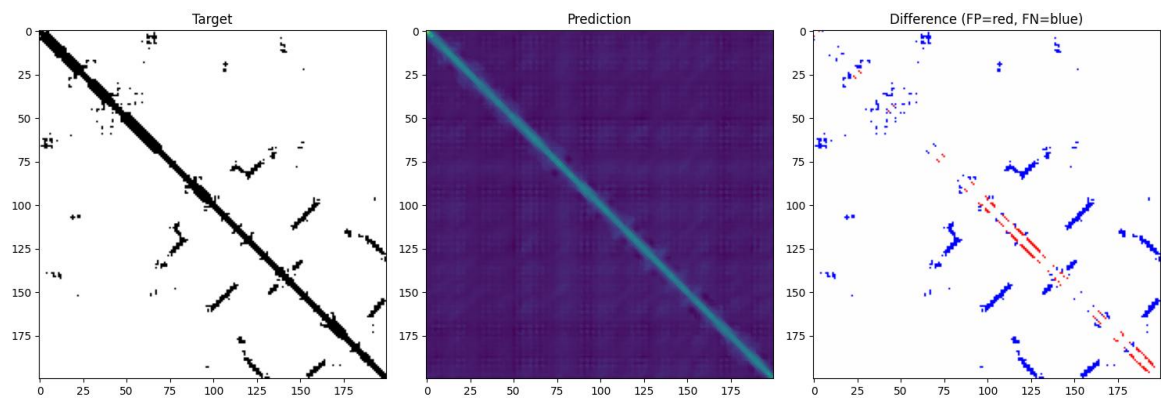


Figure 6.2: VHSE FCN Trained for 10 epochs. Target, confidence values, and difference maps. Of note is the nature of the prediction contact map and difference map, showing no correlation to the target except for a bright diagonal.

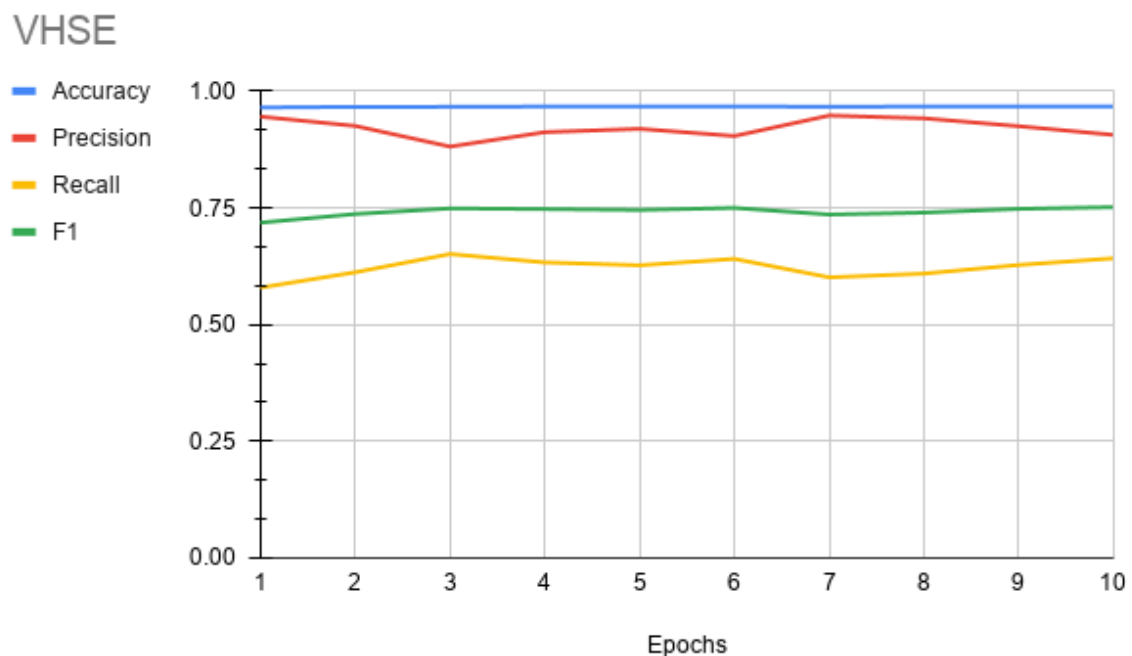


Figure 6.3: A graph showing the 4 major performance statistics of the FCN training on VHSE embeddings. You can see how despite the extremely poor performance, due to the diagonal and the sparsity of contacts away from the diagonal, almost all statistics are misleadingly high. Recall and F1 serve as the most accurate numerical measurement of these FCNs.

6.2 ProtTrans

With the baseline set from VHSE, it is apparent that ProtTrans performs better. The FCN trained on ProtTrans embeddings is able to predict the shape of novel contact maps far more accurately, see Fig. 6.5. Some detail is lost in the final prediction, but the map showing the predicted

confidence values of a contact shows a clear connection with the target contact map. Statistics reinforce this information, with ProtTrans achieving relatively high recall and F1 scores in the final epoch, at 0.82 and 0.87 respectively, see Fig. 6.7.

The increased performance comes at a cost. With the very large per-residue vector length, training the ProtTrans FCN took almost 20 times longer per epoch, at almost a full two hours. In the case of this project, that almost entirely offsets the fast encoding time when compared to PSI-BLAST. However, on a machine that is able to utilize multi-threading, or a machine with more drives, or more powerful CPU, it may be possible that the time increase from the increased vector size can be minimized by better leveraging the extremely efficient vector processing of modern graphics computing chips.

With ProtTrans successfully performing as a tertiary structure embedding, time is the primary trade off presented by the results in this project. ProtTrans has extremely quick embeddings, but far worse training times. An MSA embedding like PSI-BLAST has slow embeddings but much quicker training times. In this project, PSI-BLAST would have likely been faster, though that is not guaranteed. With the evidence of the loss chart (Fig. 6.8) and the confidence maps (Fig. 6.6) it is possible that the ProtTrans FCN only needs one or two training epochs. This in combination with the optimizations mentioned above could help reduce the large training times of the current implementation, shifting ProtTrans to being the faster embedding.

Despite this, there is still promise in the future for ProtTrans and other pLMs. In systems that already use extremely large vector MSA embeddings, ProtTrans could present a massive improvement in efficiency. Similarly, in more powerful systems where training time has a smaller impact, ProtTrans embeddings would also be a large improvement over MSA embeddings for time per-protein.

With the availability and speed of generation for pLM embeddings, if a model was made that was able to achieve a level of accuracy feasible for use in medicine research, the model could be distributed and ran locally very quickly and easily. While ProtTrans makes for a long training time, the efficiency of embeddings generation makes for an easy user experience, and the availability of the models means widespread use is feasible. This creates potential openings for new protein tertiary structure software that could compete alongside similar software.

With more development pLMs may also be able to recover more information per-residue than modern MSA embeddings, as language models and neural networks are popular specifically because of their ability to recognize difficult patterns. PLMs may also be able to reduce the size of their outputs, or others may discover ways to shrink the vectors without reducing performance.

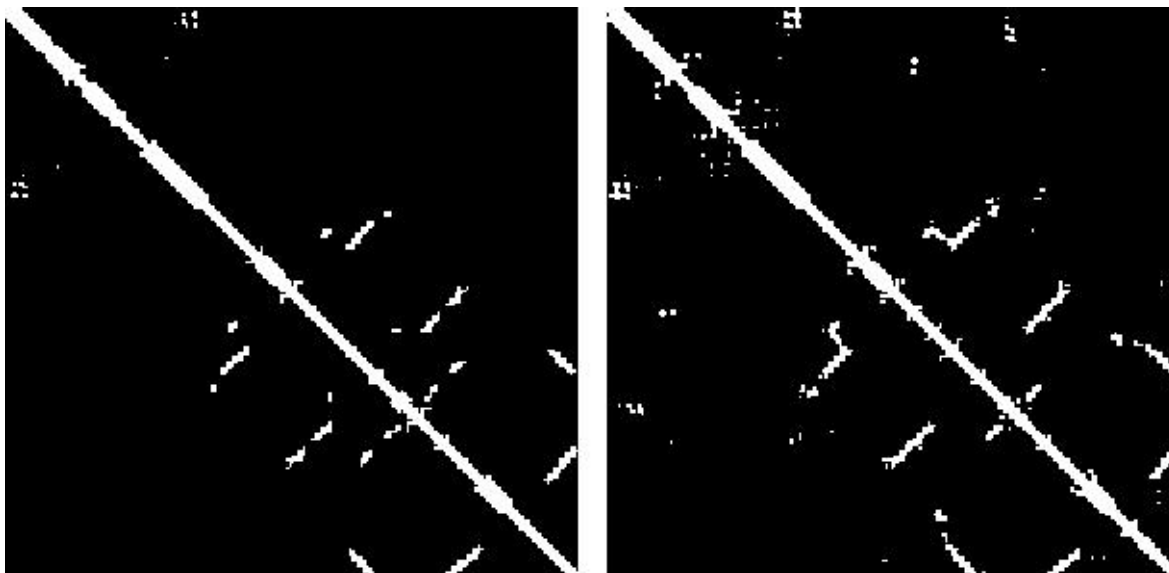


Figure 6.4: FCN Trained on ProtTrans data for 10 epochs. Predicted contact map on the left, target on the right. The detail around the major contact points is lost, but the contact map shows promise for more rigorous training.

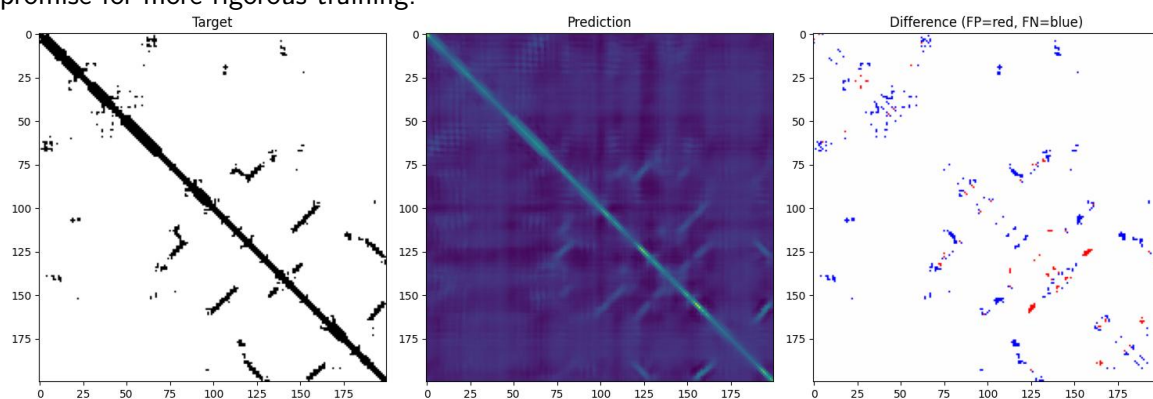


Figure 6.5: FCN Trained on ProtTrans data for 10 epochs. Target, confidence values, and difference maps. The ProtTrans data is able to much more accurately predict where contacts are likely as compared to the VHSE data. The prediction matrix shows an understanding of the protein shape, and the difference map's more balanced false positive and false negative statistics show a near-optimal FCN for a dataset of this size.

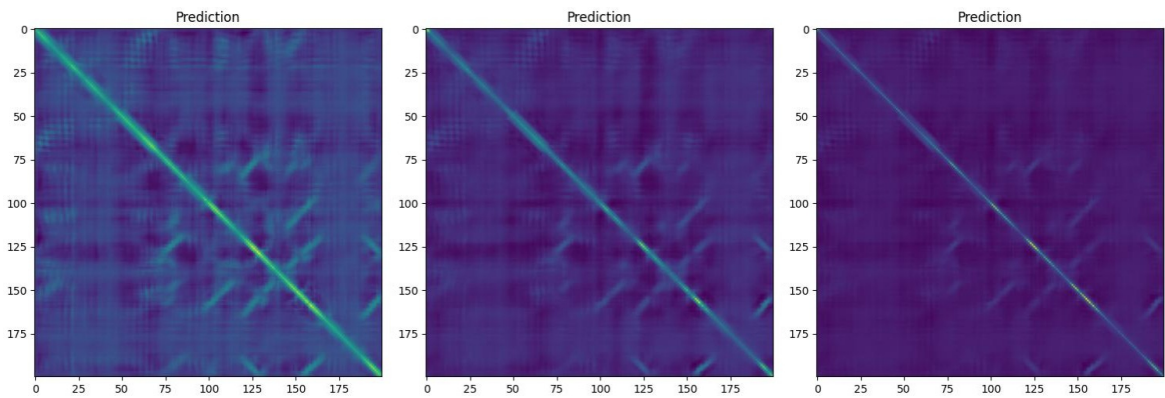


Figure 6.6: FCN Trained on ProtTrans prediction values at 1, 5, and 10 epochs. The fading of the farther contacts are likely evidence of overfitting beginning towards 10 epochs. 5 epochs maintains a strong prediction for negative and positive. Despite this, the 10th epoch does perform ever so slightly better in recall and F1, but at a loss of precision. It's likely the 10th epoch is around or just past the edge of capabilities for this particular CNN on this dataset.

Prot-Trans

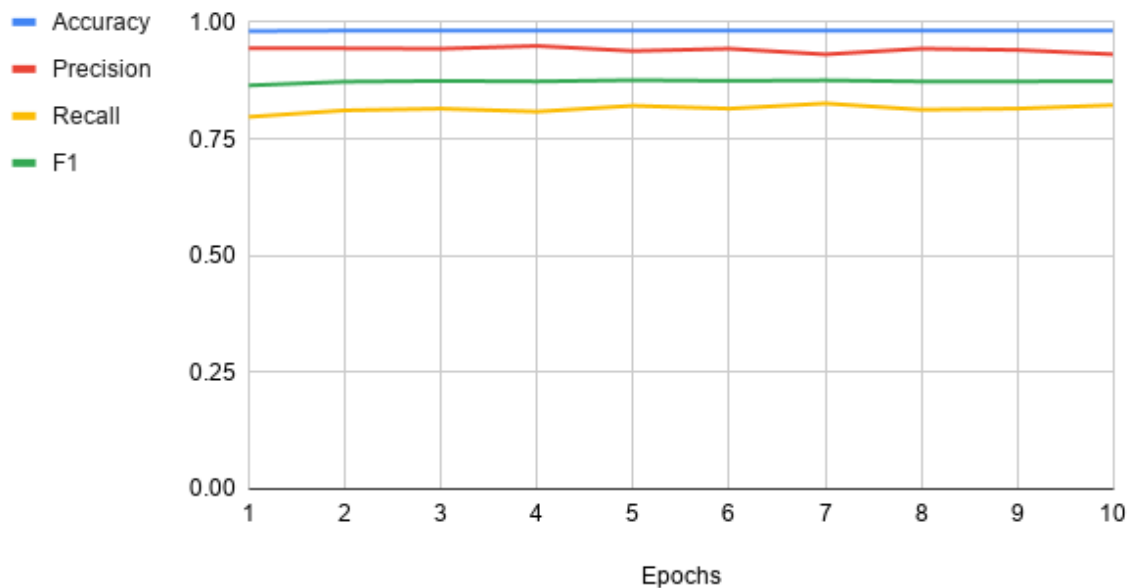


Figure 6.7: A graph showing the 4 major performance statistics of the FCN trained on ProtTrans embedding. Not only are the values significantly higher than those of the VHSE FCN, but they are also more stable later in the training cycle. These statistics do show an issue in the implementation of this project, There is not a clear or steady rise over time with training, likely meaning the improvements made over time are marginal.

Loss over Time

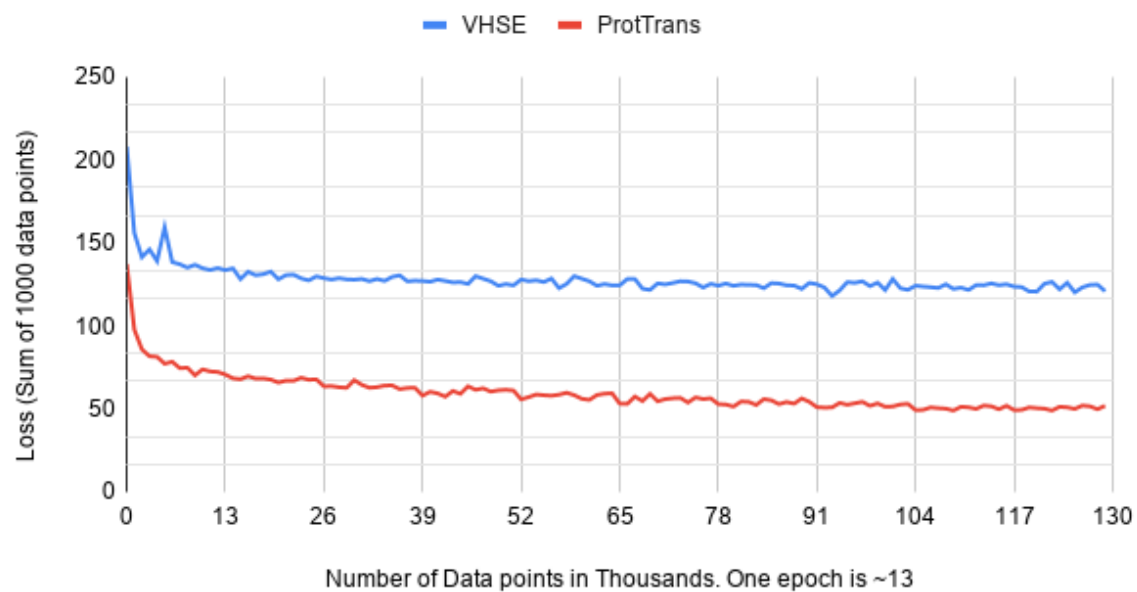


Figure 6.8: The calculated loss through each epoch. The VHSE line learns the basic connections quickly, but then entirely plateaus, showing it has likely learned all it can gain from the VHSE dataset within one epoch. The ProtTrans line maintains a much more steady downward movement after the initial drop, but still begins a plateau around epoch 7.

Chapter 7: Conclusion

Overall, the results support the viability of protein language model embeddings for some cases. Protein language models like ProtTrans' T5 model shows promise consistent with their claims. The per-residue embeddings are able to support accurate near-contact map prediction. With ProtTrans showing remarkable improvement over a baseline encoded with VHSE when used to train a supervised fully convolutional neural network, showing an increase in almost all measurements, alongside much closer visual improvements. It is clear that ProtTrans are capable protein amino acid residue embeddings.

With ProtTrans scoring far higher in both recall and F1 scores (Fig. 6.7) and achieving contact maps that are able to pick up more details in tertiary structure (Fig. 6.6) compared to VHSE, it shows promise. Similarly, its encoding time was far shorter than an MSA analog, and maintained a consistent average of 0.05 seconds per protein embedding, see Fig. 5.1.

There was a pervasive problem throughout the project regarding tradeoffs in time. For ProtTrans, this manifested as the trade off between embedding time and training time. Their embeddings are far faster to generate than their MSA analogs, but with the large dimensional size of the residue embedding vectors, the time to train a neural network increased significantly. With this problem current pLMs are likely more efficient only when the training of the neural network accounts for a minority of the total computation time. For this project, due to limitations around time and compute resources, ProtTrans is likely not faster than PSI-BLAST.

Future work could improve on the limitations posed by lack of computational resources faced in this project, as well as performing a more direct comparison between ProtTrans and PSI-BLAST. With a more efficient FCN and training loop, pLMs have the potential to replace MSAs. With more development pLMs could revolutionize the current state of protein tertiary structure prediction by allowing future tertiary structure prediction software to be far quicker and more independent. A theoretical application could use the fast ProtTrans embeddings then feed the embeddings into a local model, allowing for a faster, more accessible protein tertiary structure prediction application.

To view the applications code and data sets, the UCD CS Gitlab is available [here](#).

Acknowledgements

I would like to thank my supervisor, Professor Gianluca Pollastri, for overseeing this project and for giving me the opportunity to explore a topic at the intersection of machine learning and bioinformatics. I will likely continue working on this project for a while, as I would love to continue researching more into bioinformatics!

Thank you, Vivek Nallur and Jim Quinn, for providing general advice regarding machine learning and neural networks.

I would also like to thank my peers and family Ema-Elena Florea, Andrew Segrave, Trevor Koenig, and Taisia Danilova for helping with FCN experimentation and proof reading, as well as supporting me through this project and degree.

The gantt chart, Fig. [4.1](#), was created by Claude Sonnet 4 with a given list of dates and topic labels.

Bibliography

1. Elnaggar, A. *et al.* ProtTrans: Toward Understanding the Language of Life Through Self-Supervised Learning. *IEEE Transactions on Pattern Analysis and Machine Intelligence* **44**, 7112–7127 (2022).
2. Stoller, E. & Smith, D. Uncovering protein structure. *National Library of Medicine* (2020).
3. PN, B. & J., O. Proteins that switch folds. *Curr Opin Struct Biol* (2010).
4. Bahi, J., Bienia, W., Côté, N. & Guyeux, C. Is protein folding problem really a NP-complete one ? First investigations. *Journal of Bioinformatics and Computational Biology* (2013).
5. Martin, A., Bau, D., Vullo, A., Walsh, I. & Pollastri, G. Long-range information and physicality constrains improve predicted protein contact maps (2008).
6. Martin, A., Mooney, C., Walsh, I. & Pollastri, G. Contact Map Prediction by Machine Learning. *Wiley-Interscience* (2009).
7. Jones, B. & Shaun, K. High precision in protein contact prediction using fully convolutional neural networks and minimal sequence features. *Bioinformatics* (2018).